

DistributedDataAccessLab

V2 - Second Deliverable

E-Commerce Marketplace Backend

Alexandre Schneider

FDU MSACS - CSCI 6844 V1
Programming for the Internet

April 2026

V2 - Second Deliverable Changelog

Commit: 5a6bdbb - V2: RabbitMQ messaging, API Gateway (YARP), DTOs, docker-compose update

Date: April 6, 2026

Stats: 33 files changed, +1,251 lines, -341 lines

A) Asynchronous Messaging with RabbitMQ (MassTransit)

Objective: Asynchronous communication between microservices using a message broker.

New files

File	Description
OrderService/.../Contracts/Events.cs	Event contracts (OrderCreatedEvent, OrderCancelledEvent, OrderItemEvent)
NotificationService/.../Contracts/Events.cs	Same contracts replicated (shared namespace Contracts.Events)
NotificationService/.../Consumers/OrderCreatedConsumer.cs	Consumer: receives OrderCreatedEvent, creates notifications for buyer & sellers
NotificationService/.../Consumers/OrderCancelledConsumer.cs	Consumer: receives OrderCancelledEvent, notifies buyer & sellers

Modified files

File	Change
OrderService/.../Program.cs	Added AddMassTransit() with RabbitMQ config
OrderService/.../OrderService.Api.csproj	Added MassTransit.RabbitMQ 8.4.0 package
OrderService/.../OrdersController.cs	Injected IPublishEndpoint; Create publishes OrderCreatedEvent; Cancel publishes OrderCancelledEvent
NotificationService/.../Program.cs	Added AddMassTransit() with AddConsumer<OrderCreatedConsumer> and AddConsumer<OrderCancelledConsumer>
NotificationService/.../csproj	Added MassTransit.RabbitMQ 8.4.0 package

Event Flow

- Buyer creates an order via POST /api/orders
- OrdersController saves the order and publishes OrderCreatedEvent to RabbitMQ
- OrderCreatedConsumer in NotificationService receives the event and creates notifications for buyer and each seller
- Same flow for cancellation: OrderCancelledEvent -> OrderCancelledConsumer

B) API Gateway with YARP

Objective: Single entry point for all microservices + data aggregation endpoint.

New project: ApiGateway

File	Description
ApiGateway.csproj	.NET 10 project - Yarp.ReverseProxy 2.3.0, Swashbuckle 10.1.2
Program.cs	YARP reverse proxy setup + named HttpClient for aggregation + Swagger
appsettings.json	YARP routes for 6 microservices + clusters with Docker hostnames
Controllers/AggregateController.cs	GET /api/aggregate/order-details/{orderId} - calls Order, Customer, Product in parallel
Dockerfile	Multi-stage build following existing services pattern

YARP Routes

Route	Destination
/api/customers/{**catch-all}	http://customerservice:8080
/api/products/{**catch-all}	http://productservice:8080
/api/orders/{**catch-all}	http://orderservice:8080
/api/notifications/{**catch-all}	http://notificationsservice:8080
/api/cart/{**catch-all}	http://cartservice:8080
/api/payments/{**catch-all}	http://paymentservice:8080

Aggregated Endpoint

GET /api/aggregate/order-details/{orderId} returns combined JSON with { order, customer, products } from 3 services in parallel.

C) DTOs in All Services

Objective: Never expose domain entities directly in APIs; use request/response DTO records.

New DTO files (Dtos/ folder in each service)

Service	DTOs Created
CustomerService	CreateCustomerDto, UpdateCustomerDto, CustomerResponseDto, CustomerDetailDto, CustomerDashboardDto, PersonalDashboardDto, CustomerSummaryDto
OrderService	CreateOrderRequestDto, CreateOrderItemDto, UpdateStatusDto, CancelOrderDto, ReturnOrderDto, OrderResponseDto, OrderItemResponseDto, OrderDashboardDto, StatusCountDto, BuyerDashboardDto, LastOrderDto, SellerDashboardDto
ProductService	CreateProductDto, UpdateProductDto, StockUpdateDto, StockDecrementDto, DiscountUpdateDto, ProductResponseDto, ProductStockDto, ProductDiscountDto, ProductDashboardDto, CategoryStatsDto, SellerDashboardDto, SellerProductDto
CartService	AddCartItemDto, UpdateCartItemDto, CartResponseDto, CartItemResponseDto, EmptyCartDto, CheckoutResponseDto
PaymentService	CreatePaymentDto, PaymentResponseDto, PaymentProcessResultDto, PaymentDashboardDto, PaymentStatusStatsDto, PaymentMethodStatsDto
NotificationService	CreateNotificationDto, NotificationResponseDto, NotificationPageDto, UnreadCountDto, NotificationDashboardDto, TypeCountDto, UserUnreadDto, MarkReadResultDto

Controllers modified (all 6)

Controller	Changes
CustomersController.cs	All endpoints return DTOs; Create/Update accept DTOs; added MapToDto()
OrdersController.cs	Same + removed old inline classes; deleted Models/Dtos.cs
ProductsController.cs	Same + removed inline StockUpdateRequest, StockDecrementRequest, DiscountUpdateRequest
CartController.cs	Same + removed inline request classes
PaymentsController.cs	Same + removed inline CreatePaymentRequest
NotificationsController.cs	Same pattern applied

D) Updated docker-compose.yml

Services added

Service	Config
rabbitmq	Image: rabbitmq:3-management-alpine, Ports: 5672/15672, Healthcheck: rabbitmq-diagnostics -q ping
apigateway	Build: ApiGateway/ApiGateway, Port: 5000->8080, depends_on all 6 microservices, YARP env overrides

Services modified

Service	Change
orderservice	Added RabbitMQ__Host: rabbitmq + depends_on rabbitmq (service_healthy)
notificationservice	Added RabbitMQ__Host: rabbitmq + depends_on rabbitmq (service_healthy)

E) Other Changes

File	Change
DistributedDataAccessLab.sln	Added ApiGateway project to solution
.gitignore	Added rules for *.db, *.db-shm, *.db-wal
CustomerService/.../Dockerfile	Minor build adjustment
README.md	Rewritten with service table, V2 section, updated tech stack

Final Architecture (V2)

